

VISION-TO-CART

MCP Commerce Intelligence Platform

Architecture & Implementation Plan · Hackathon 3.0

V4 — SIMPLIFIED FOR HACKATHON

SCOPE: HACKATHON · TIMEBOX: 8 HOURS

■ V4 Scope Decisions — Approved Architecture (Read Before Building)

Layer	Status	Reason / Action
Data & Services Layer	■ New — build fully	Zone 1 backbone. Product Catalog DB, Vector DB (ChromaDB), Redis, Elasticsearch, API Gateway. All tools access data through gateway only.
Error Recovery	■ Mandatory — build fully	The one layer judges will see live if anything fails. Retry × 2 + fallback tool + graceful response.
Frontend / React Client	✗ Out of scope	Removed from 8-hour build. Demo via demo_test.py or Postman hitting /mcp/stream directly.
Confidence Governance	Simplified only	Single <0.5 threshold. Show 'Here are some alternatives.' No full routing tree.
Feedback & Learning Loop	✗ Document only	No demo-visible impact. Use static popularity_score from catalog.json instead.
Observability Dashboard	✗ Minimal only	Console logs + /metrics JSON endpoint (~15 lines). No dashboard.
Security Module	Inline only	4 input guards inline in main.py. No separate security.py.
Full Confidence Routing Tree	✗ Document only	High/Medium/Low 3-tier routing not built. Simplified threshold only.

■ Mandatory Deliverables (8-Hour Scope)

End-to-End Pipeline: Image + Text Discovery → Top Results

Simplified Confidence Handling (single 0.5 threshold)

Error Recovery: Retry + Fallback + Graceful Response

Docker Compose: One-Command Startup

3 Demo Scenarios with Expected Outputs

Priority build order: catalog JSON + real sample file → 3 demo scenarios with expected outputs → Data Services Layer + Redis → Error Recovery → core pipeline → simplified confidence threshold → /metrics endpoint.

Ship the working pipeline first. The bones of this plan are excellent — the risk is over-engineering into a late integration.

1. Solution Overview

The platform enables users to discover eyewear products through three mechanisms that converge into a unified Product Intelligence Core:

- Image-Based Product Discovery — upload a real-world photo, get matched products
- Natural Language Product Discovery — celebrity names, movie references, style descriptions
- Conversational Product Exploration — follow-up refinements using session memory

Both image and NL paths produce a StyleDNA object that feeds the same retrieval, ranking, and recommendation pipeline — consistent results regardless of input mode. All data access flows through the Data Services API Gateway.

2. High-Level Architecture — V4 Three Zones

Zone 1 — Data & Services Layer (NEW in V4 — Primary Backbone)

- Product Catalog DB — products, categories, attributes
- Vector Database (ChromaDB) — embeddings generation & semantic search
- Knowledge Graph — brand, style, similarity relationships
- Cache (Redis) — session store, response caching
- Search Index (Elasticsearch) — keyword search fallback
- File Storage (S3 / local) — images, documents
- Session Store (Redis) — session DB across requests
- Data Services API Gateway — unified access: auth, rate limiting, validation, caching

Zone 2 — MCP Server (Orchestration Layer)

- MCP Orchestrator (Brain) — Intent Router, Context Manager, Plan Executor, Response Composer
- Tool Router + Workflow Manager + Memory Manager
- Cross-Cutting: Simplified Confidence Governance · Error Recovery (mandatory) · Security (basic) · Observability (basic)
- SSE /mcp/stream endpoint — heartbeat every 3 seconds

Zone 3 — Tools (Execution Layer)

- Image Intelligence Tool — preprocessing + YOLOv8 + GPT-4o vision → StyleDNA
- NL Understanding Tool — intent extraction + Tavily/Brave web search → StyleDNA
- Catalog Intelligence Tool — enrichment, style classification, vectorization & indexing
- Search & Retrieval Tool — hybrid search (vector + keyword), filtering, Top-N
- Ranking & Scoring Tool — 5-component score, confidence check, explainability
- Recommendation Tool — budget / premium / trending alternatives
- PDP / Cart / Checkout Tool — mock PDP, Add-to-Cart, cart session

3. MCP Orchestration Layer

Component	Responsibility
Intent Router	Classifies incoming request as IMAGE_SEARCH or NL_SEARCH. Routes to correct pipeline.
Context Manager	Maintains session state. Detects refinement signals ('cheaper', 'same brand') and merges with prior context.

Plan Executor	Builds ordered sub-task list for the pipeline. Emits step_start / step_complete SSE events for each.
Response Composer	Assembles final result_ready SSE event from ranked ProductMatch list + recommendation strip.
Tool Router	Sole dispatcher to Zone 3 tools. Entry point for Error Recovery wrapping every tool call.
Workflow Manager	Coordinates execution flow across all discovery pipelines.
Memory Manager	Conversational continuity: 'show cheaper options', 'different color', 'same brand'. Backed by Redis.
Confidence Governance	Simplified: if confidence < 0.5 → return 'Here are some alternatives' flag. Full 3-tier routing is post-hackathon.
Error Recovery	■ MANDATORY. Retry × 2 with exponential backoff (1s, 3s) → fallback tool → graceful partial response.

4. Data & Services Layer — Implementation Detail

The API Gateway is a thin FastAPI router that sits between all tools and all data stores. Every tool calls the gateway — never a store directly. This is the key V4 architectural addition.

Service	Technology (Hackathon)	Role
Product Catalog DB	catalog.json (30+ products)	Products, categories, attributes, popularity scores
Vector DB	ChromaDB (in-process)	CLIP embeddings, metadata indexing, keyword indexing
Knowledge Graph	In-memory dict	Brand, style, similarity relationships
Cache	Redis (Docker)	Session store, response caching for repeated queries
Search Index	Elasticsearch (Docker, optional)	Keyword search fallback when vector search confidence is low
File Storage	Local /data/ directory	Product images, uploaded user images
API Gateway	FastAPI middleware	Unified access: validation, rate limiting, caching, response standardisation

■ **Docker Compose starts the full stack with one command**

Services: mcp_server (port 8000) + redis:7-alpine (port 6379) + elasticsearch:8 (port 9200, optional)

ChromaDB runs in-process — no Docker container needed, persistent directory mount only.

Run: docker compose up → then: python scripts/demo_test.py

5. Image Discovery Pipeline (4A)

Objective: Convert an uploaded image into structured product intelligence.

1. Receive image (base64 encoded)
2. Preprocess — background removal via rembg + resize to 640×640

- 3. Object detection — YOLOv8n crops eyewear region; Grounding DINO as text-prompted fallback
- 4. Attribute extraction — GPT-4o vision (paid) or LLaVA 1.6 (free) returns structured JSON
- 5. Normalize attributes through normalization dictionary ('golden'→'gold', 'roundish'→'round')
- 6. Generate StyleDNA (4A StyleDNA)
- 7. Pass StyleDNA → Product Intelligence Core (4C)

Output Field	Example Value
Extracted Signals	Product Type · Shape · Frame Style · Color · Material · Style Indicators
StyleDNA Output	shape='aviator', frame_color='gold', lens_color='green', material='metal', style_tags=['classic','luxury'], confidence=0.87, source='image'
Object Detection	YOLOv8n primary + Grounding DINO text-prompted fallback — keep as designed, well-thought-out

6. Natural Language Discovery Pipeline (4B)

Objective: Convert natural language requests into structured product intelligence.

- 1. Receive query + session context
- 2. Intent schema extraction — classify as Celebrity / Product / Style / Trend / Recommendation search
- 3. Query decomposition — break into 3–5 targeted sub-queries
- 4. Multi-source web search — Tavily (primary) / Brave Search (fallback)
- 5. Evidence validation — reject single-source results; require 2+ corroborating sources
- 6. StyleDNA synthesis from validated evidence (4B StyleDNA)
- 7. Pass StyleDNA → Product Intelligence Core (4C)

Intent Type	Example Query	Web Search?
Celebrity Search	'Akshay Khanna sunglasses in Dhurandhar'	Yes — actor + movie
Product Search	'Minimalist gold frame glasses'	No — direct attributes
Style Search	'Retro round frames like the 70s'	Optional — trend research
Trend Search	'What celebs wore at Cannes 2025'	Yes — event research
Recommendation	'Show me something premium'	No — catalog filter

7. Product Intelligence Core — 4C Unified

Both 4A and 4B pipelines produce a StyleDNA object and pass it here. All ranking, confidence, and recommendation logic is written once and shared by both pipelines.

Catalog Intelligence Service

Component	What it does
Attribute Enrichment	Adds style taxonomy tags, trend scores, gender affinity to each catalog product
Style Classification	Assigns each product to taxonomy class: classic-luxury / casual-everyday / sport-performance / fashion-forward / minimalist-clean

Trend Signals	Numeric 0–100 trend score assigned manually for hackathon (oversized, tinted, thin metal = trending)
Similar Product Search	DOCUMENT ONLY — skip pre-computation. Use live embedding similarity at query time instead. Post-hackathon extension.
Metadata Generation	Builds to_search_text() natural-language sentence per product for embedding

Vectorization & Indexing

Index Type	Tool	Purpose
CLIP Embeddings	ChromaDB + sentence-transformers / text-embedding-3-small	Semantic similarity search
Metadata Index	ChromaDB metadata filters	Hard filters: price range, gender, product type
Keyword Index	Elasticsearch (optional)	Keyword fallback when embedding confidence is low

Retrieval & Ranking

Stage	Detail
Hybrid Search	Vector similarity (ChromaDB cosine) + keyword search (Elasticsearch) combined
Filtering	Pre-filter by assets, brand, price range, gender before similarity scoring
Ranking Score	5-component: Embedding Similarity 40% · Shape Match 20% · Brand Match 15% · Lens Color 15% · Metadata Match 10%
Tie-ranking	Cross-encoder re-ranking improves final ordering for close scores
Top-N Results	Top 4 returned as ProductMatch objects with confidence score, match reason, and score breakdown

8. Result Enrichment & Decision Layer

Explainability Engine

For every ProductMatch, compare StyleDNA attributes against product attributes and build a human-readable match reason. Display on every product card / JSON result.

- Example: "Matched because: ✓ Aviator shape ✓ Green lens ✓ Thin metal frame — Score: 91%"

Confidence Engine — Simplified Implementation

■ Implement this simplified version only — not the full 3-tier routing

After ranking, check top result's score against a single threshold:

- confidence ≥ 0.5 → Return results as-is. Show score normally.
- confidence < 0.5 → Return results with message: "Here are some alternatives we think you might like."

One if-statement. 20 minutes to implement. Visually identical to full routing from judge's perspective.

■ Post-Hackathon: Full 3-Tier Routing (document in README, do not build now)

High 90–100%: Direct match, green badge, no supplementary search.

Medium 70–89%: Return result + trigger supplementary attribute search, show 'strong alternative' note.

Low <70%: De-emphasise primary, trigger full fallback chain, promote fallback results.

Fallback Search Strategy (Decision Engine)

Executed in order, stopping at first non-empty result:

- 1. Attribute Search — query directly by shape + style tags (bypasses vector store)
- 2. Style Search — query by style_tags only
- 3. Similar Product Search — query by taxonomy class
- 4. Popular Products fallback — always returns something

9. Output & Experience Layer

Output	Detail
Top Products (with scores)	Top-4 ranked products with confidence scores, match explanations, and score breakdowns
Product Detail Page	Mock PDP endpoint per product: image, attributes, price, Add-to-Cart button
Recommendations	Budget / premium / trending strip below main results
Add to Cart & Checkout	Cart managed as session state in Redis. Returns cart contents and running total.
Real-time Streaming	SSE /mcp/stream endpoint. Step-by-step progress events. Heartbeat every 3 seconds.

Frontend (React client) is OUT OF SCOPE for the 8-hour build. All output is verified via scripts/demo_test.py hitting the /mcp/stream SSE endpoint directly. The JSON output of each demo scenario is the deliverable, not a UI.

10. The 3 Required Demo Scenarios

Define these with expected outputs in inputs/test_queries.txt and scripts/demo_test.py **before starting the build**. Run all three via `python scripts/demo_test.py`.

Scenario	Input	Expected Output
1 — Image Search	Real-world photo of aviator sunglasses — person wearing them, busy background	Top result: aviator shape, confidence ≥ 0.7 , match reason shows shape + lens color match
2 — NL Celebrity Search	"Sunglasses worn by Akshay Khanna in Dhurandhar"	Web search triggered, StyleDNA synthesized from evidence, top result matches extracted style profile
3 — Conversational Refinement	"Show me cheaper options" (same session_id as Scenario 2)	Web search skipped (context from Redis), price filter applied, budget-tier products returned

11. Error Recovery — Mandatory Implementation

■ Build this fully — it is the only mandatory V3/V4 addition

This is the one layer judges will see live if anything fails during the demo.

Every tool call in the Tool Router is wrapped in this 3-layer strategy.

Layer	Trigger	Action
Layer 1 — Retry	Transient error: network timeout, API rate limit	Retry up to 2 times. Wait 1s after first failure, 3s after second.
Layer 2 — Fallback Tool	Retries exhausted	Activate tool-specific fallback: Tavily fails → Brave Search. GPT-4o fails → GPT-4o mini / LLaVA.
Layer 3 — Graceful Response	Fallback also fails	Return partial result — never an error. NL web search fails → fall back to direct style-tag catalog search on query text. User always sees products.

12. Cross-Cutting Layers — Applied Across All Zones

Layer	Status	What to Build
Security (Basic)	Inline in main.py	4 guards: validate session_id charset [a-zA-Z0-9-], reject payloads >10MB, reject empty queries, wrap user text in <user_input> delimiters before LLM calls. No security.py file.
Observability (Basic)	~15 lines only	Console logs per tool call + /metrics GET endpoint returning in-memory JSON of last request's per-tool timing and success/failure status.
Confidence Governance	Simplified only	Single if confidence < 0.5 check in confidence_governance.py. Return 'alternatives' flag. Full 3-tier routing is documented in README as post-hackathon extension.
Memory & Context	Build fully	Redis-backed session state. Context Manager detects refinement signals and merges with prior StyleDNA. Essential for Scenario 3.
Scalability	Design only	Modular, stateless services architecture is the design. Actual load testing is post-hackathon.

13. Files to Change — V4 Quick Reference

File / Path	Change	Priority
<ParagraphStyle 'c2'>		
File / Path	Change	Priority
data_services/api_gateway.py	NEW (V4). Scaffold unified data access layer: validation, rate limiting, caching middleware.	P1

data_services/catalog/catalog.json	NEW. 30+ products. Remove similarity_graph field. Add: style_taxonomy, trend_score, popularity_score.	P1
scripts/ingest_catalog.py	NEW. One-time script: embed all products into ChromaDB. Must complete before demo.	P1
scripts/demo_test.py	NEW. Automated API calls for all 3 demo scenarios. Prints pass/fail.	P1
mcp_server/error_recovery.py	BUILD FULLY. 3-layer: retry(×2, 1s/3s backoff) → fallback tool → graceful partial response.	P1
mcp_server/main.py	Inline 4 security guards at SSE endpoint. Wire /metrics endpoint. No separate security.py.	P1
tools/image_intelligence_tool.py	RENAMED from image_tool.py. Full pipeline: rembg → YOLOv8 → GPT-4o extraction → StyleDNA.	P1
tools/nl_understanding_tool.py	RENAMED + MERGED from web_search_tool.py. Intent extraction + Tavily/Brave + evidence validation + StyleDNA.	P1
tools/catalog_intelligence_tool.py	NEW. Attribute enrichment, style classification, vectorization & indexing via API Gateway.	P2
tools/search_retrieval_tool.py	RENAMED from search_tool.py. Hybrid vector + keyword search, filtering, Top-N via API Gateway.	P2
tools/ranking_scoring_tool.py	RENAMED from ranking_tool.py. Remove medium-confidence supplementary search trigger.	P2
tools/pdp_cart_checkout_tool.py	NEW. Mock PDP lookup + Add-to-Cart + cart session management via Redis.	P2
mcp_server/confidence_governance.py	Replace 3-tier routing tree with single <0.5 threshold. Keep full spec as commented block.	P3
mcp_server/observability.py	~15 lines: record(tool, duration_ms, success) + get_metrics(). Wire to /metrics GET in main.py.	P3
docker-compose.yml	Add redis:7-alpine + elasticsearch:8 (optional) services alongside mcp_server.	P1
README.md	Add 'Planned Post-Hackathon Extensions' section documenting: Feedback Loop, full Confidence Routing, Observability Dashboard, Security Module, Similarity Graph, Embedding Drift Mitigation.	P3

14. Minor Notes from Review — Keep As-Is

Item	Note
YOLO + Grounding DINO fallback	Well-thought-out. YOLOv8n primary + Grounding DINO text-prompted fallback when YOLO finds no eyewear. Keep exactly as designed.

SSE heartbeat every 3 seconds	Right call for perceived performance. A single NL query involves 3–5 sequential LLM calls (20–30s total). Heartbeat makes it feel active. Keep it.
Docker Compose single-command startup	Essential. Do not skip. Any setup friction during demo costs marks. docker compose up must start everything.

15. Evaluation Checklist — V4

Run through this in the 30 minutes before demo recording. Every unchecked item is a lost mark.

Architecture & MCP (20 marks)

- Data Services API Gateway scaffolded — all tools access data only through gateway
- Redis running and session state persisting correctly across at least 3 turns
- docker compose up starts full stack with one command
- Tool Router is the only entry point to tool execution — orchestrator never calls tools directly
- SSE /mcp/stream emits at least 5 distinct labeled step events before final result
- Error recovery demonstrated: show a graceful fallback trigger in demo scenario
- Session isolation verified: two different session_ids cannot access each other's context

Image Pipeline (20 marks)

- Background removal (rembg) applied before attribute extraction
- YOLOv8n object detection crops eyewear from real-world photo
- Grounding DINO fallback activates when YOLO finds nothing
- Normalization dictionary applied — 'golden' and 'gold' both match the same products
- Hybrid 5-component scoring formula used — score breakdown in JSON output
- Confidence <0.5 → 'Here are some alternatives' message in response
- Explainability output in every ProductMatch: match reason with matched attributes listed
- Fallback search chain triggered with deliberately ambiguous test image

NL Pipeline (15 marks)

- Intent schema extracted and logged (visible in SSE step events or terminal)
- Query decomposed into 3+ distinct search sub-queries
- Evidence validation step rejects single-source results
- StyleDNA synthesized and included in JSON response
- Scenario 3 (conversational refinement) resolves using Redis session context, no re-search

Product Mapping (15 marks)

- catalog.json has 30+ products with full attribute coverage across all shapes/colors/materials

■	All products embedded into ChromaDB (verified via startup log)
■	Top-3 results always returned — no empty result set in any scenario
■	All products return a PDP endpoint response with image, attributes, price
■	Add-to-Cart updates Redis cart session and returns updated cart contents

Code Quality (10 marks)

■	Each tool file has exactly one tool class with one clear responsibility
■	No tool file imports from any other tool file
■	All API keys in .env — no hardcoded values anywhere
■	README documents every env var with free-tier alternatives noted
■	inputs/ contains at least 5 test images and 5 test queries
■	docker-compose.yml starts full stack with one command
■	README has 'Planned Post-Hackathon Extensions' section

Demo Clarity (5 marks)

■	All 3 scenarios run cleanly via demo_test.py without errors
■	Architecture overview given at start of demo (zones, cross-cutting layers, mandatory deliverables)
■	Error recovery demonstrated live at least once
■	/metrics endpoint briefly shown as architectural maturity signal
■	No hardcoded or pre-computed results — every demo result is computed live

End of Vision-to-Cart Implementation Plan — V4 Simplified for Hackathon

Full V4 architecture diagram: HLD_diagram_hackathon_v4.png

Extended specifications (post-hackathon layers): Vision-to-Cart_V4_Hackathon.md

Scope: Hackathon · Timebox: 8 Hours · Success = Working Pipeline + Clear Demo + Smooth Experience